



Exploring the Impact of Different Types of AI Assistance in Beginner-Level Programming: Productivity Booster or Learning Hindrance?

Group37: Xuning Hu(2251992), Tiantian Ye(2251663), Yiyang Han(2252155), Lekun Liu(2255714), Yichuan Zhang(2251668)

Abstract

AI-powered code generators have become a common tool to support novice programmers in completing coding tasks. These tools aim to improve both productivity and learning by generating code directly or indirectly from natural language descriptions. While prior work has examined the impact of AI assistance on code quality and user memory [1], less is known about how different types of AI assistance affect novice users' efficiency and learning outcomes. In this study, we classify AI code assistance into four conditions: (1) no AI assistance, (2) direct code-level suggestions (e.g., Copilot), (3) AI-generated code explanations followed by manual modification, and (4) fully generated code solutions. Each participant completed two similar tasks: the first to assess productivity under each condition, and the second to evaluate learning gains. Our results show that full-code generation yields the highest task efficiency and the lowest perceived workload (NASA-TLX), while explanation-based assistance leads to the highest learning gains and the top post-task performance rankings. These findings suggest a trade-off between productivity and learning across different AI assistance types and offer implications for designing AI tools for novice programmers.

01 Research Questions

- RQ1:** Can AI assistance improve task efficiency for novice programmers, and which type of AI assistance is the most effective in doing so?
- RQ2:** Can AI assistance enhance learning outcomes for novice programmers, and which type of AI assistance leads to the greatest learning gains?

03 User Study and Evaluation Metrics

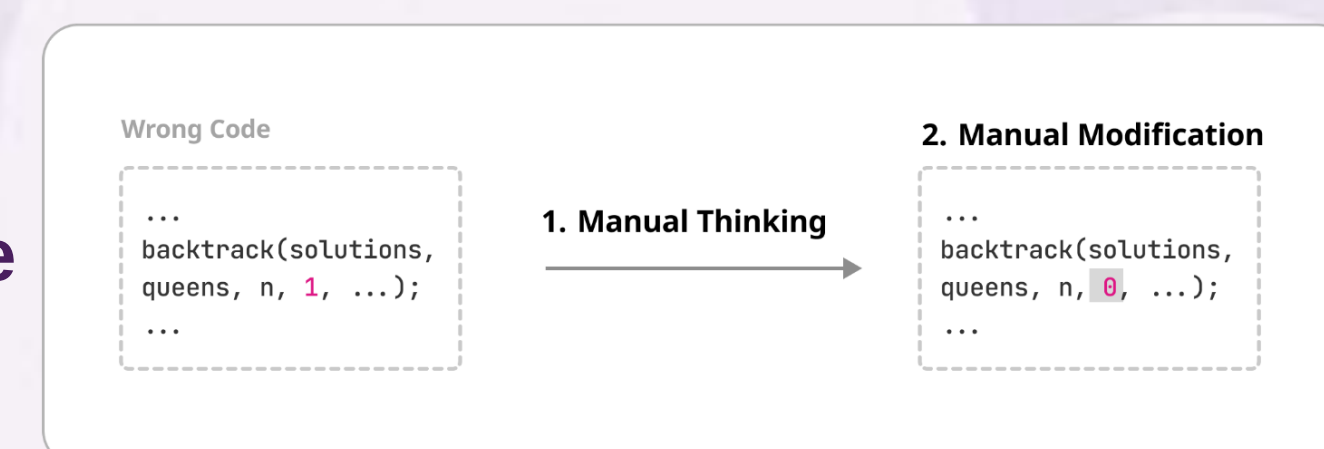
We conducted two programming tasks to evaluate the effectiveness of different AI-assisted coding techniques. A total of 16 participants (11 male, 5 female), aged between 19 and 25 ($M = 20.6$, $SD = 1.52$), were recruited from various majors at a local university, including Information and Computer Science, Electronic Engineering, and Digital Media Technology. Participants' self-reported familiarity with programming was measured using a 7-point Likert scale ($M = 1.72$, $SD = 0.84$). This study followed a between-subjects design, with four participants assigned to each of the four AI assistance conditions. In the first task, participants used their assigned AI technique to modify a piece of code. In the second task, they were asked to complete a similar coding task without any AI assistance, to evaluate what they had learned.

- Task Efficiency:** The time taken to complete the first task measured how efficiently participants used the AI tool.
- Learning Outcomes:** The second task's completion time (without AI assistance) was used to indicate participants' learning.
- Perceived Workload (NASA-TLX) [2]:** To measure the task load induced by the techniques with 7-point scales.
- Preference Ranking and Interview:** Participants ranked the four AI techniques based on their preferences

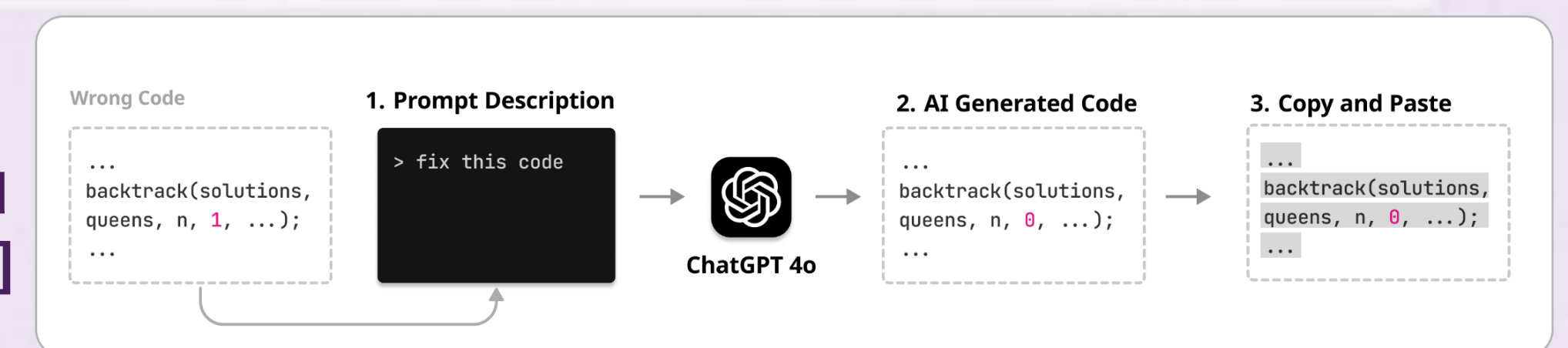
02 Design and Methodology:

- No AI assistance:** Users are required to search for relevant knowledge online and manually revise their code based on the issues they encounter.
- AI-generated code explanations:** Users request explanations from the AI assistant to understand the code and manually make the necessary changes.
- AI fully generated code solutions:** Users obtain complete code solutions from the AI assistant and copy them to complete the task.
- Direct code-level suggestions:** Users invoke an AI assistant to directly modify the code within the code editor.

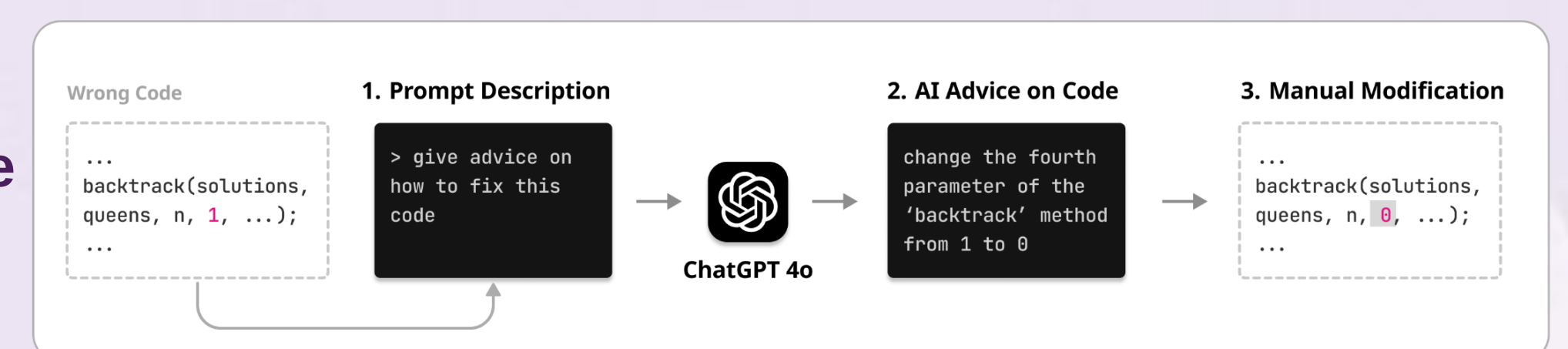
No AI assistance



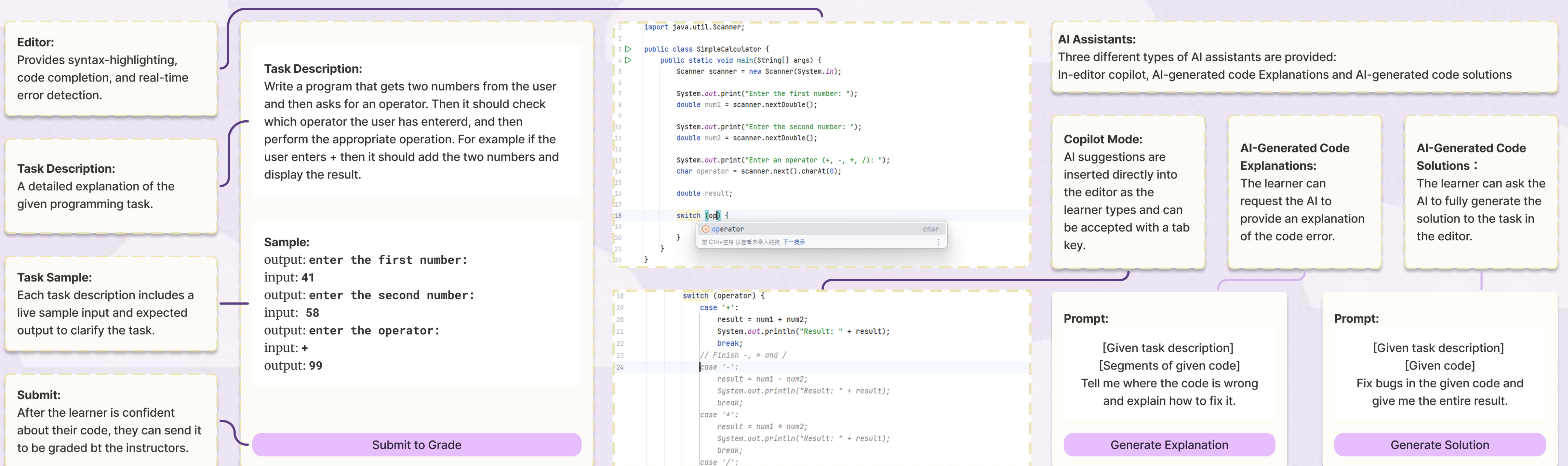
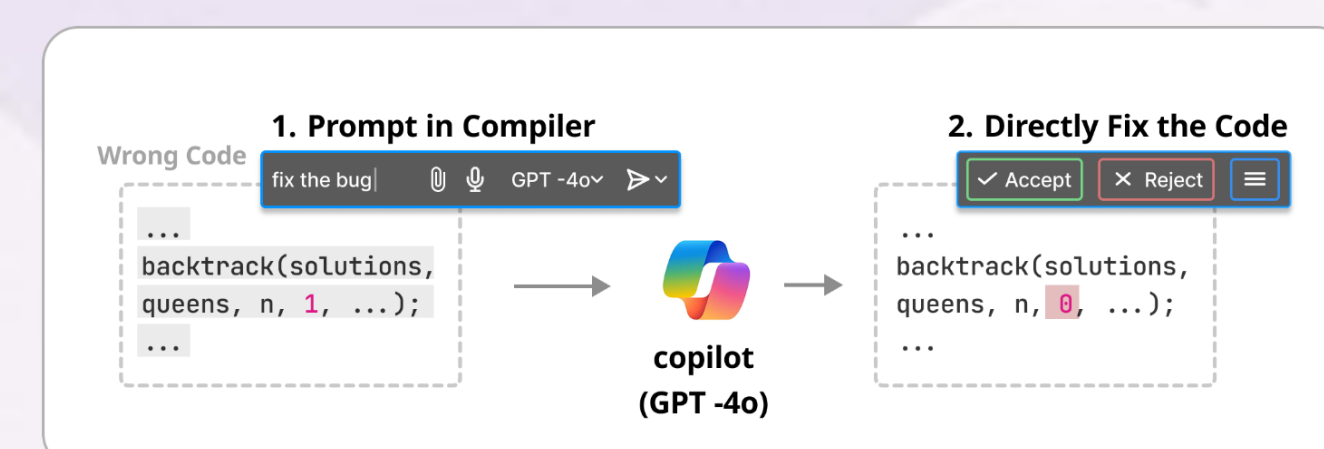
AI fully generated code solutions [3]



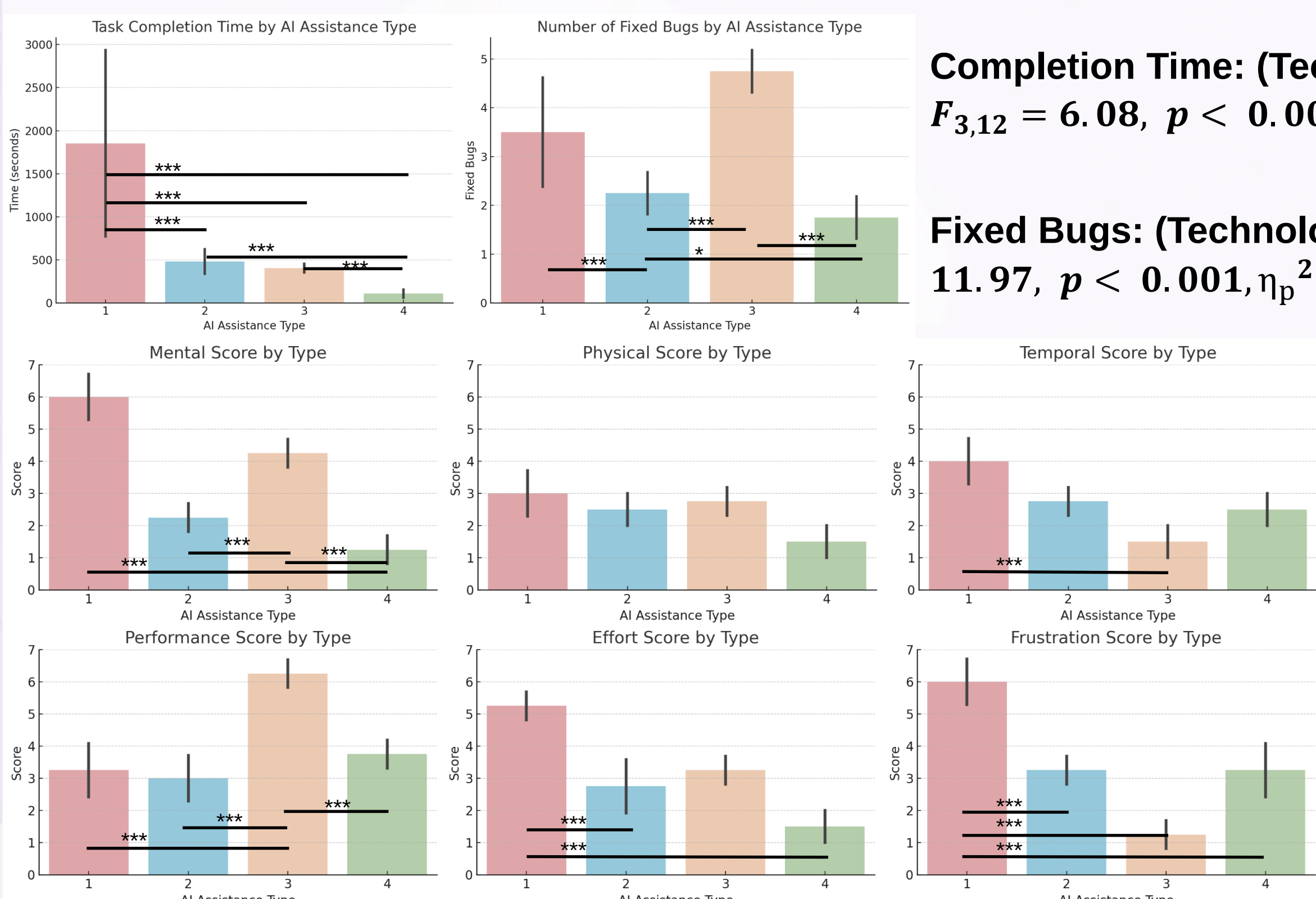
AI-generated code explanations [3]



Direct code-level suggestions [4]



04 Results



Completion Time: (Technology: $F_{3,12} = 6.08$, $p < 0.001$, $\eta_p^2 = 0.603$)

Fixed Bugs: (Technology: $F_{3,12} = 11.97$, $p < 0.001$, $\eta_p^2 = 0.749$)

Reference

- [1] Kazemitabaar, M., Hou, X., Henley, A., Ericson, B. J., Weintrop, D., & Grossman, T. (2024). *How Novices Use LLM-based Code Generators to Solve CS1 Coding Tasks in a Self-Paced Learning Environment*. In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research (Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, Article 3, 1–12. <https://doi.org/10.1145/3631802.3631806> [2] X. Hu, W. Xu, Y. Wei, H. Zhang, J. Huang and H. -N. Liang, "Optimizing Moving Target Selection in VR by Integrating Proximity-Based Feedback Types and Modalities," *2025 IEEE Conference Virtual Reality and 3D User Interfaces (VR)*, Saint Malo, France, 2025, pp. 52-62, doi: 10.1109/VR59515.2025.00030. [3] R. Xiao, X. Hou, and J. Stamper, "Exploring How Multiple Levels of GPT-Generated Programming Hints Support or Disappoint Novices," in *Extended Abstracts of the CHI Conf. on Human Factors in Computing Systems (CHI EA '24)*, Honolulu, HI, USA, 2024, Article 142, 10 pages. Available: <https://doi.org/10.1145/3613905.3650937> [4] Dakhel A M, Majdinasab V, Nikanjam A, et al. Github copilot ai pair programmer: Asset or liability?[J]. *Journal of Systems and Software*, 2023, 203: 111734.

05 Discussion

Exploring the Impact of Different Types of AI Assistance on User Efficiency (RQ1):

- AI-generated complete code was the most time-efficient method.
- Copilot-based code modification and AI-generated code explanations also improved efficiency compared to no AI, but the difference between the two was not significant.
- User 3:** "Without AI assistance, I had to manually filter through information online, which was very time-consuming."
- Users 5 & 6:** "Using code explanation techniques added more cognitive load and took more time."

Exploring the Impact of Different Types of AI Assistance on Learning Outcomes (RQ2):

- AI-generated explanations followed by manual code editing led to the best learning outcomes (most bugs fixed), though differences were not statistically significant.
- Full code generation by AI was the most efficient, but resulted in the poorest learning outcomes.
- User 4:** "After the AI generated the complete code, my first reaction was to try it out in the compiler. If it didn't work, I would just ask the AI to revise it again. I didn't really learn anything."